

human

What information do you have on roofs in Exeter

host_agent

Calling tool generate_metadata_for_all_artifacts with args : {} Tool generate_metadata_for_all_artifacts returned result : {}

Calling tool send_message with args : {'target': 'planning_agent', 'task_description': 'User wants information on roofs in Exeter. What are the general steps to solve this query using available geospatial agents and datasets?'}

planning_agent

The area specified is Exeter, so there will be 1 entry for Exeter. The entity to search for is roofs, but it is unclear what specific information the user is looking for regarding roofs (e.g., types, materials, services, etc.). I need to clarify this with the user before proceeding.

1. Ask the user to clarify what specific information they are looking for regarding roofs in Exeter.

Output steps: ["Find Exeter 1 area", "Ask user for clarification on specific information about roofs in Exeter"]

host_agent

Tool send_message returned result : The area specified is Exeter, so there will be 1 entry for Exeter. The entity to search for is roofs, but it is unclear what specific information the user is looking for regarding roofs (e.g., types, materials, services, etc.). I need to clarify this with the user before proceeding.

1. Ask the user to clarify what specific information they are looking for regarding roofs in Exeter.

Output steps: ["Find Exeter 1 area", "Ask user for clarification on specific information about roofs in Exeter"]

Calling tool send_message with args : {'target': 'named_area', 'task_description': 'Find the area polygon for Exeter.'}

named_area

Calling tool call_os_ngd with args : {'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_area_polygon'} Tool call_os_ngd returned result : ['Search results have been found. \n Artifact generated is : exeter_area_polygon,\n Description is : A geopandas dataframe

containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter..,\n count of records fetched : 98.\n ',]

Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'exeter_area_polygon' to find the specific polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'."}

data_analysis_agent

Calling tool generate_metadata_for_artifacts with args : {'artifact_names': ['exeter_area_polygon']} Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter using multiple columns to ensure accuracy
    filtered = gdf[(gdf['name1_text'].str.lower() == 'exeter') |
                   (gdf['description'].str.contains('Exeter', case=False, na=False)) |
                   (gdf['theme'].str.contains('Exeter', case=False, na=False))]
    # Further refine: prefer exact match in name1_text
    filtered = filtered[filtered['name1_text'].str.lower() == 'exeter']
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter."
    return [summary, 'filtered_exeter_area_polygon', 'GeoDataFrame containing only the polygon(s) for Exeter, filtered using name1_text and description columns']
```

Tool code_executor returned result : ['Filtered to 1 polygon(s) for Exeter.',]

Filtered to 1 polygon for Exeter. - Artifact name: filtered_exeter_area_polygon - Description: GeoDataFrame containing only the polygon(s) for Exeter, filtered using name1_text and description columns for high accuracy.

named_area

The specific polygon for Exeter has been filtered and saved as:

- **Artifact Name:** filtered_exeter_area_polygon
- **Description:** GeoDataFrame containing only the polygon(s) for Exeter, filtered using name1_text and description columns for high accuracy.

host_agent

Calling tool send_message with args : {'target': 'buildings', 'task_description': 'Search for roof information within the filtered_exeter_area_polygon artifact. Focus on any available data related to roofs in Exeter, such as roof types, materials, or other roof-specific attributes.'}

buildings

Calling tool call_os_ngd with args : {'filters': None, 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': False, 'filename': 'exeter_roof_data'} Tool call_os_ngd returned result : ["Multiple search results have been found in multiple datasets. A summary of each is provided.\n Artifacts generated are : ['buildings_exeter_roof_data', 'buildingline_exeter_roof_data', 'buildingpart_exeter_roof_data'],\n Descriptions are : ['A geopandas dataframe containing building data with filters and bbox applied for the query Search for roof information within the filtered_exeter_area_polygon artifact. Focus on any available data related to roofs in Exeter, such as roof types, materials, or other roof-specific attributes. using filters None.', 'A geopandas dataframe containing building line data with filters and bbox applied for the query Search for roof information within the filtered_exeter_area_polygon artifact. Focus on any available data related to roofs in Exeter, such as roof types, materials, or other roof-specific attributes. using filters None.', 'A geopandas dataframe containing building part data with filters and bbox applied for the query Search for roof information within the filtered_exeter_area_polygon artifact. Focus on any available data related to roofs in Exeter, such as roof types, materials, or other roof-specific attributes. using filters None.'],\n counts of records fetched are : [69169, 69342, 85480].\n ", [, ,]]

Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Analyze the artifacts 'buildings_exeter_roof_data', 'buildingline_exeter_roof_data', and 'buildingpart_exeter_roof_data' to extract information related to roof types, materials, and other roof-specific attributes. Save the filtered results as 'filtered_roof_info_exeter'."}

data_analysis_agent

Calling tool generate_metadata_for_artifacts with args : {'artifact_names': ['buildings_exeter_roof_data', 'buildingline_exeter_roof_data', 'buildingpart_exeter_roof_data']} Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
import pandas as pd

def extract_roof_info_exeter(data: list):
    # Unpack the dataframes
    buildings, buildingline, buildingpart = data

    # Select relevant roof columns from buildings, only those that exist
```

```

roof_columns_buildings = [
    'osid', 'description', 'roofmaterial_primarymaterial',
    'roofshapeaspect_shape', 'geometry'
]
buildings_roof = buildings[[col for col in roof_columns_buildings if col in buildings.columns]]

# For buildingpart, select roof-related columns if present
roof_columns_buildingpart = [
    'osid', 'toid', 'description', 'height_absoluteroofbase_m', 'height_relativeroofbase_m',
    'height_absolutemax_m', 'height_relativemax_m', 'height_absolutemin_m',
    'height_confidencelevel', 'geometry', 'geometry_area_m2'
]
buildingpart_roof = buildingpart[[col for col in roof_columns_buildingpart if col in buildingpart.columns]]

# For buildingline, keep description and geometry
buildingline_roof = buildingline[['osid', 'toid', 'description', 'geometry']]

# Add source column
buildings_roof['source'] = 'buildings'
buildingpart_roof['source'] = 'buildingpart'
buildingline_roof['source'] = 'buildingline'

# Add missing columns for concatenation
for df in [buildings_roof, buildingpart_roof, buildingline_roof]:
    if 'toid' not in df.columns:
        df['toid'] = None
    if 'roofmaterial_primarymaterial' not in df.columns:
        df['roofmaterial_primarymaterial'] = None
    if 'roofshapeaspect_shape' not in df.columns:
        df['roofshapeaspect_shape'] = None

# Reorder columns for consistency
columns_order = [
    'osid', 'toid', 'description', 'roofmaterial_primarymaterial',
    'roofshapeaspect_shape', 'source', 'geometry'
]
all_roof_info = pd.concat([
    buildings_roof[columns_order],
    buildingpart_roof[columns_order],
    buildingline_roof[columns_order]
], ignore_index=True)

summary = f"Extracted roof-related information from 3 datasets. Total entries: {len(all_roof_info)}."
artifact_name = 'filtered_roof_info_exeter'
artifact_description = (
    "A combined dataframe containing roof-specific attributes (type, material, shape, etc.) "
    "from buildings, building parts, and building lines in Exeter. Includes source information and geometry."
)

```

```
return [summary, artifact_name, artifact_description, all_roof_info]
```

Tool code_executor returned result : ['Extracted roof-related information from 3 datasets. Total entries: 223991.',]

Extracted roof-related information from 3 datasets. Total entries: 223991. An artifact has been generated with the name 'filtered_roof_info_exeter' and contains a combined dataframe of roof-specific attributes (type, material, shape, etc.) from buildings, building parts, and building lines in Exeter. The artifact includes source information and geometry for each entry.

buildings

The search for roof information within the "filtered_exeter_area_polygon" has been completed. The following artifact has been generated:

- **Artifact Name:** filtered_roof_info_exeter
- **Description:** A combined dataframe containing roof-specific attributes (type, material, shape, etc.) from buildings, building parts, and building lines in Exeter. Includes source information and geometry.

This artifact contains detailed roof-related information extracted from the datasets.

host_agent

I have found detailed information on roofs in Exeter. The available data includes:

- Roof-specific attributes such as type, material, shape, and more.
- This information is extracted from buildings, building parts, and building lines within the Exeter area.

If you need a summary, visualization, or specific details (like roof types, materials, or distribution), please let me know how you'd like to proceed!

human

show me all information on roofs in Exeter

host_agent

Calling tool send_message with args : {'target': 'planning_agent', 'task_description': 'User wants all information on roofs in Exeter. Provide the general steps to solve this geospatial query.'}

planning_agent

The area is Exeter, so there is 1 entry for Exeter. The entity is roofs, which may refer to various aspects such as types of roofs, roofing services, or roof-related regulations. Since the query is broad, I need to clarify what specific information about roofs the user is looking for.

1. Identify the general area: Exeter (1 entry). 2. Identify the entity: roofs (need to clarify what specific information is required). 3. Ask the user for clarification on what specific information they want regarding roofs (e.g., types, services, regulations).

Output steps: 1. Find Exeter 1 area. 2. Ask the user to clarify what specific information they want about roofs in Exeter.

host_agent

Tool send_message returned result : The area is Exeter, so there is 1 entry for Exeter. The entity is roofs, which may refer to various aspects such as types of roofs, roofing services, or roof-related regulations. Since the query is broad, I need to clarify what specific information about roofs the user is looking for.

1. Identify the general area: Exeter (1 entry). 2. Identify the entity: roofs (need to clarify what specific information is required). 3. Ask the user for clarification on what specific information they want regarding roofs (e.g., types, services, regulations).

Output steps: 1. Find Exeter 1 area. 2. Ask the user to clarify what specific information they want about roofs in Exeter.

Calling tool generate_metadata_for_all_artifacts with args : {} Tool generate_metadata_for_all_artifacts returned result : {}

Calling tool send_message with args : {'target': 'named_area', 'task_description': 'Find the area polygon for Exeter.'}

named_area

Calling tool call_os_ngd with args : {'filters': None, 'bbox': None, 'polygon_or_point': True, 'street_address': None, 'filename': 'exeter_area_polygon'} Tool call_os_ngd returned result : ['Search results have been found. \n Artifact generated is : exeter_area_polygon,\n Description is : A geopandas dataframe containing named area data with no filters applied for the query (so remember no search is performed here so spurious entities present) Find the area polygon for Exeter..,\n count of records fetched : 98.\n ',]

Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Filter the artifact 'exeter_area_polygon' to find the specific polygon for Exeter. Save the filtered results as 'filtered_exeter_area_polygon'."}

data_analysis_agent

Calling tool generate_metadata_for_artifacts with args : {'artifact_names': ['exeter_area_polygon']} Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
def filter_exeter_area_polygon(data: list):
    gdf = data[0]
    # Filter using multiple columns to ensure we get the correct polygon for Exeter
    filtered = gdf[(gdf['name1_text'].str.lower() == 'exeter') |
                   (gdf['description'].str.contains('Exeter', case=False, na=False)) |
                   (gdf['theme'].str.contains('exeter', case=False, na=False))]
    # Further refine: prefer exact matches in name1_text
    filtered_exact = filtered[filtered['name1_text'].str.lower() == 'exeter']
    if not filtered_exact.empty:
        filtered = filtered_exact
    summary = f"Filtered to {len(filtered)} polygon(s) for Exeter."
    artifact_name = 'filtered_exeter_area_polygon'
    artifact_description = 'GeoDataFrame containing only the polygon(s) specifically for Exeter, filtered using name and description columns.'
    return [summary, artifact_name, artifact_description, filtered]
```

Tool code_executor returned result : ['Filtered to 1 polygon(s) for Exeter.',]

Filtered to 1 polygon for Exeter. The results are saved as 'filtered_exeter_area_polygon', which contains only the polygon specifically for Exeter, filtered using the name and description columns.

named_area

The specific area polygon for Exeter has been filtered and saved as 'filtered_exeter_area_polygon'.

host_agent

Calling tool send_message with args : {'target': 'buildings', 'task_description': "Search for all buildings in the area defined by 'filtered_exeter_area_polygon'. Focus on extracting information related to roofs or roof types if available."}

buildings

Calling tool call_os_ngd with args : {'filters': None, 'bbox': 'filtered_exeter_area_polygon', 'polygon_or_point': True, 'street_address': False, 'filename': 'exeter_buildings_data'} Tool call_os_ngd returned result : ['Multiple search results have been found in multiple datasets. A summary of each is provided.\n Artifacts generated are : ['buildings_exeter_buildings_data', 'buildingline_exeter_buildings_data', 'buildingpart_exeter_buildings_data'],\n Descriptions are : ["A geopandas dataframe containing building data with filters and bbox applied for the query Search for all buildings in the area defined by 'filtered_exeter_area_polygon'. Focus on extracting information related to roofs or roof types if available. using filters None.", "A geopandas dataframe containing building line data with filters and bbox applied for the query Search for all buildings in the area defined by 'filtered_exeter_area_polygon'. Focus on extracting information related to roofs or roof types if available. using filters None.", "A geopandas dataframe containing building part data with filters and bbox applied for the query Search for all buildings in the area defined by 'filtered_exeter_area_polygon'. Focus on extracting information related to roofs or roof types if available. using filters None."],\n counts of records fetched are : [69169, 69342, 85480].\n ', [, ,]]

Calling tool send_message with args : {'target': 'data_analysis_agent', 'task_description': "Analyze the artifacts 'buildings_exeter_buildings_data', 'buildingline_exeter_buildings_data', and 'buildingpart_exeter_buildings_data' to extract information related to roofs or roof types. Save the filtered results as 'exeter_roof_data'."}

data_analysis_agent

Calling tool generate_metadata_for_artifacts with args : {'artifact_names': ['buildings_exeter_buildings_data', 'buildingline_exeter_buildings_data', 'buildingpart_exeter_buildings_data']} Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
import pandas as pd
def extract_exeter_roof_data(data: list):
    # Unpack the dataframes
    buildings, buildingline, buildingpart = data
    # --- Extract roof-related columns from buildings ---
    roof_columns_buildings = [
        'osid', 'description', 'roofmaterial_primarymaterial', 'roofmaterial_solarpanelpresence',
        'roofmaterial_greenroofpresence', 'roofmaterial_confidenceindicator',
        'roofmaterial_evidencedate', 'roofmaterial_updatedate', 'roofmaterial_capturemethod',
        'roofshapeaspect_shape', 'roofshapeaspect_areapitched_m2', 'roofshapeaspect_areaflat_m2',
        'roofshapeaspect_areafacingnorth_m2', 'roofshapeaspect_areafacingnortheast_m2',
        'roofshapeaspect_areafacingeast_m2', 'roofshapeaspect_areafacingsoutheast_m2',
        'roofshapeaspect_areafacingsouth_m2', 'roofshapeaspect_areafacingsouthwest_m2',
        'roofshapeaspect_areafacingwest_m2', 'roofshapeaspect_areafacingnorthwest_m2',
```



```
        'roofshapeaspect_areaindeterminable_m2', 'roofshapeaspect_areatotal_m2',
        'roofshapeaspect_confidenceindicator', 'roofshapeaspect_evidencedate',
        'roofshapeaspect_updatedate', 'roofshapeaspect_capturemethod', 'geometry'
    ]
    roof_buildings = buildings[roof_columns_buildings]
    # --- Extract roof-related columns from buildingpart ---
    roof_columns_buildingpart = [
        'osid', 'toid', 'description', 'height_absoluteroofbase_m', 'height_relativeroofbase_m',
        'height_absolutemax_m', 'height_relativemax_m', 'height_absolutemin_m',
        'height_confidencelevel', 'height_evidencedate', 'height_updatedate', 'geometry'
    ]
    roof_buildingpart = buildingpart[roof_columns_buildingpart]
    # --- No roof info in buildingline, skip ---
    # --- Combine results ---
    roof_buildings['source'] = 'buildings'
    roof_buildingpart['source'] = 'buildingpart'
    # Standardize columns for concatenation
    roof_buildingpart = roof_buildingpart.reindex(columns=roof_buildings.columns, fill_value=None)
    exeter_roof_data = pd.concat([roof_buildings, roof_buildingpart], ignore_index=True)
    summary = f"Extracted roof-related data from buildings and buildingpart. Total entries: {len(exeter_roof_data)}."
    artifact_name = 'exeter_roof_data'
    artifact_description = 'Filtered data from buildings and buildingpart datasets in Exeter, containing roof material, roof shape, and related attributes.'
    return [summary, artifact_name, artifact_description, exeter_roof_data]
```

Tool code_executor returned result : ['Extracted roof-related data from buildings and buildingpart. Total entries: 154649.',]

Extracted roof-related data from buildings and buildingpart. Total entries: 154649. - Artifact name: exeter_roof_data - Description: Filtered data from buildings and buildingpart datasets in Exeter, containing roof material, roof shape, and related attributes.

buildings

The search and analysis have been completed. Here are the results:

- **Artifact Name:** exeter_roof_data
- **Description:** Filtered data from buildings and building part datasets in Exeter, containing roof material, roof shape, and related attributes.

This dataset contains the roof-related information you requested.

host_agent

Calling tool send_message with args : {'target': 'plotting_agent', 'task_description': "Plot the artifact 'exeter_roof_data' to show all roof-related information (roof material, roof shape, and related attributes) for buildings in Exeter."}

plotting_agent

Calling tool generate_metadata_for_artifacts with args : {'artifact_names': ['exeter_roof_data']} Tool generate_metadata_for_artifacts returned result : ...

Tool Code Executor

```
def plot_exeter_roof_data(data: list):
    import folium
    import geopandas as gpd
    import matplotlib
    import matplotlib.colors as mcolors
    import numpy as np

    gdf = data[0].copy()
    # Convert CRS to EPSG:4326 for folium
    if gdf.crs is not None and gdf.crs.to_epsg() != 4326:
        gdf = gdf.to_crs(epsg=4326)

    # Prepare color maps for roof material and shape
    roof_materials = gdf['roofmaterial_primarymaterial'].fillna('Unknown').unique()
    roof_shapes = gdf['roofshapeaspect_shape'].fillna('Unknown').unique()

    material_colors = dict(zip(roof_materials, mcolors.TABLEAU_COLORS))
    shape_colors = dict(zip(roof_shapes, mcolors.CSS4_COLORS))

    # Create folium map centered on Exeter
    centroid = gdf.geometry.unary_union.centroid
    m = folium.Map(location=[centroid.y, centroid.x], zoom_start=14, tiles='cartodbpositron')

    # Add polygons colored by roof material and popup with all roof info
    for idx, row in gdf.iterrows():
        color = material_colors.get(row['roofmaterial_primarymaterial'], '#3186cc')
        shape_color = shape_colors.get(row['roofshapeaspect_shape'], '#3186cc')
        popup_html = f"""
        <b>Description:</b> {row['description']}<br>
        <b>Roof Material:</b> {row['roofmaterial_primarymaterial']}<br>
        <b>Solar Panel:</b> {row['roofmaterial_solarpanelpresence']}<br>
        <b>Green Roof:</b> {row['roofmaterial_greenroofpresence']}<br>
        <b>Roof Shape:</b> {row['roofshapeaspect_shape']}<br>
        <b>Area Pitched (m2):</b> {row['roofshapeaspect_areapitched_m2']}<br>
        <b>Area Flat (m2):</b> {row['roofshapeaspect_areaflat_m2']}<br>
        <b>Total Roof Area (m2):</b> {row['roofshapeaspect_areatotal_m2']}<br>
        """
```

```

        <b>Material Confidence:</b> {row['roofmaterial_confidenceindicator']}<br>
        <b>Shape Confidence:</b> {row['roofshapeaspect_confidenceindicator']}<br>
        <b>Evidence Date:</b> {row['roofmaterial_evidencedate']}<br>
        <b>Update Date:</b> {row['roofmaterial_updatedate']}<br>
        """
        geojson = folium.GeoJson(
            row['geometry'],
            style_function=lambda feature, color=color: {
                'fillColor': color,
                'color': shape_color,
                'weight': 1,
                'fillOpacity': 0.6
            },
            tooltip=folium.Tooltip(row['description']),
            popup=folium.Popup(popup_html, max_width=350)
        )
        geojson.add_to(m)

    # Add a legend for roof material
    legend_html = '<div style="position: fixed; bottom: 50px; left: 50px; width: 250px; height: auto; z-index:9999; background: white; border:2px solid grey; padding: 5px;">
    legend_html += '<b>Roof Material Legend</b><br>'
    for mat, col in material_colors.items():
        legend_html += f'<i style="background:{col};width:18px;height:18px;float:left;margin-right:8px;opacity:0.7;"></i>{mat}<br>'
    legend_html += '<br><b>Roof Shape Outline</b><br>'
    for shp, col in list(shape_colors.items())[:5]:
        legend_html += f'<i style="border:2px solid {col};width:18px;height:18px;float:left;margin-right:8px;"></i>{shp}<br>'
    legend_html += '</div>'
    m.get_root().html.add_child(folium.Element(legend_html))

    # Save map
    map_filename = 'exeter_roof_data_map.html'
    m.save(map_filename)

    # Prepare summary
    summary_rows = gdf[['description', 'roofmaterial_primarymaterial', 'roofmaterial_solarpanelpresence', 'roofmaterial_greenroofpresence', 'roofshapeaspect_shape', 'roofshapeaspect_areatotal_m2']]
    summary = f"Plotted {len(gdf)} buildings in Exeter with roof material, shape, and related attributes. First 5 entries:\n" + summary_rows.to_string(index=False)
    return [summary, 'exeter_roof_data_map', 'Folium map showing all Exeter buildings with roof material, shape, and related roof attributes. Polygons colored by material and shape']

```

Tool code_executor returned result : ['Plotted 154649 buildings in Exeter with roof material, shape, and related attributes. First 5 entries:\n description
roofmaterial_primarymaterial roofmaterial_solarpanelpresence roofmaterial_greenroofpresence roofshapeaspect_shape
roofshapeaspect_areatotal_m2\nDomestic Outbuilding Waterproof Membrane Or Concrete Not Present Not Present Flat 18.7\nEnd-Of-Terrace House Tile
Or Stone Or Slate Not Present Not Present Pitched 128.7\nMid-Terrace House Tile Or Stone Or Slate Not Present Not Present Pitched 103.5\n
Mid-Terrace House Tile Or Stone Or Slate Not Present Not Present Pitched 87.5\nUnknown Building Tile Or Stone Or Slate Not Present Not Present
Pitched 10.7',]

[Plotted 154649 buildings in Exeter with roof material, shape, and related attributes. First 5 entries:\n description roofmaterial_primarymaterial roofmaterial_solarpanelpresence roofmaterial_greenroofpresence roofshapeaspect_shape roofshapeaspect_areatotal_m2\nDomestic Outbuilding Waterproof Membrane Or Concrete Not Present Not Present Flat 18.7\nEnd-Of-Terrace House Tile Or Stone Or Slate Not Present Not Present Pitched 128.7\n Mid-Terrace House Tile Or Stone Or Slate Not Present Not Present Pitched 103.5\n Mid-Terrace House Tile Or Stone Or Slate Not Present Not Present Pitched 87.5\n Unknown Building Tile Or Stone Or Slate Not Present Not Present Pitched 10.7', 'exeter_roof_data_map', 'Folium map showing all Exeter buildings with roof material, shape, and related roof attributes. Polygons colored by roof material and outlined by roof shape. Popups show detailed roof info.', 'exeter_roof_data_map.html']

host_agent

All available information on roofs in Exeter has been gathered and visualized. Here’s what was found:

- Over 154,000 buildings in Exeter have been analyzed for roof-related attributes.
- The data includes:
- Roof material (e.g., tile, stone, slate, waterproof membrane, concrete)
- Presence of solar panels or green roofs
- Roof shape (e.g., flat, pitched)
- Total roof area

A map has been generated showing all buildings in Exeter, colored by roof material and outlined by roof shape. You can interact with the map to see detailed roof information for each building.

If you need a specific aspect of roof information (such as only solar panels, or only green roofs), or want a summary or statistics, please let me know!